DATA ANALYTICS PROJECT REPORT

Customer Shopping Behaviour Analysis Using Power BI

Tools Used

Python,

PostgreSQL,

SQL,

Power BI,

Excel

Technologies

Pandas,

Power Query,

DAX,

Visualization

Student Name

Vaibhav N Badiger

Project Type

Data Analytics Project

ABSTRACT

In today's competitive retail environment, understanding customer purchasing behaviour is essential for making informed business decisions. This project focuses on analysing customer shopping patterns using a combination of data analytics tools including Python, PostgreSQL, SQL, and Power BI.

The project begins with data cleaning and preprocessing using Python in a Jupyter Notebook environment. The cleaned dataset is then stored in a PostgreSQL database using SQL Alchemy. Advanced SQL queries are used to extract business insights from the data. Finally, the processed data is visualized using Power BI to build interactive dashboards.

The dashboard provides insights into customer purchasing trends, product popularity, revenue contribution, shipping preferences, and customer segmentation. The insights derived from the analysis can help businesses optimize marketing strategies, improve product offerings, and enhance customer satisfaction.

INTRODUCTION

Customer behaviour analysis is a crucial component of modern data-driven decision making. Businesses rely heavily on data analytics to understand how customers interact with their products and services. By analysing purchasing patterns, businesses can identify trends, predict future behaviour, and optimize their strategies.

This project focuses on analysing a dataset containing customer shopping behaviour information such as purchase amount, product category, review ratings, shipping type, subscription status, and purchase frequency.

The project integrates multiple technologies including Python for data preprocessing, PostgreSQL for database management, SQL for querying, and Power BI for data visualization. The final outcome is an interactive dashboard that allows users to explore customer purchasing patterns and derive meaningful business insights.

PROBLEM STATEMENT

Retail businesses often collect large volumes of customer data but struggle to convert it into actionable insights. Without proper analysis, valuable patterns related to customer preferences, spending habits, and product popularity remain hidden.

The primary objective of this project is to analyse customer purchasing behaviour and identify patterns that can help businesses improve sales strategies, marketing campaigns, and customer engagement.

PROJECT OBJECTIVES

The main objectives of this project are:

1. To clean and preprocess customer shopping data.
2. To store structured data in a relational database.
3. To analyse customer behaviour using SQL queries.
4. To identify key business insights from the dataset.
5. To develop an interactive Power BI dashboard.
6. To visualize important metrics such as revenue, purchase patterns, and customer segments.

DATASET DESCRIPTION

The dataset used in this project is **Customer Shopping Behaviour Dataset** containing **3900 customer records** and **18 attributes**.

Important attributes include:

- Customer ID
- Age
- Gender
- Item Purchased
- Category
- Purchase Amount (USD)
- Location
- Review Rating
- Subscription Status
- Shipping Type
- Discount Applied
- Previous Purchases
- Payment Method
- Frequency of Purchases

This dataset provides detailed insights into customer purchasing patterns and product preferences.

RAW DATASET

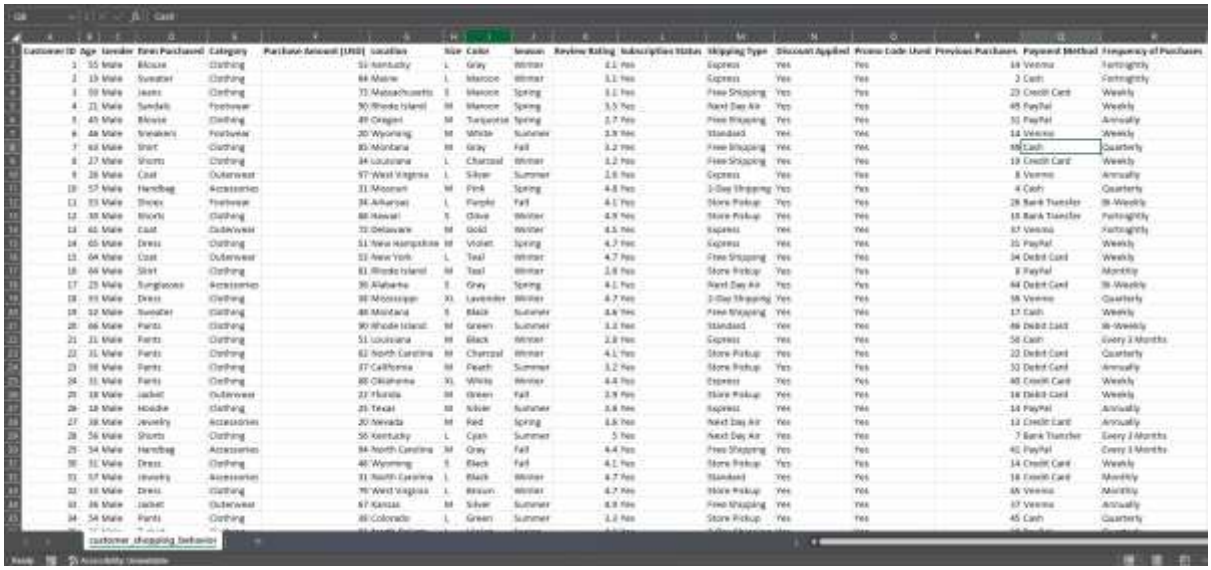


Fig 1: Dataset

TOOLS AND TECHNOLOGIES USED

The following tools were used in this project:

Python: Python was used for data preprocessing and cleaning using the Pandas library.

Jupyter Notebook: Jupyter Notebook was used to perform data exploration and transformation operations.

PostgreSQL: PostgreSQL database was used to store structured data after preprocessing.

PgAdmin: pgAdmin was used as the database management interface for executing SQL queries.

SQL: SQL was used to perform business queries and generate analytical insights.

Power BI: Power BI was used to create interactive dashboards and visualize key metrics.

DATA CLEANING AND PREPROCESSING

Data preprocessing was performed using Python in Jupyter Notebook.

The following steps were carried out:

Handling Missing Values

Missing values in the Review Rating column were filled using the median rating of each category.

This ensured that rating values remained realistic within each product category.

Column Name Standardization

Column names were converted into **snake_case format** to make them more consistent and database-friendly.

Example:

Customer ID → customer_id
Purchase Amount (USD) → purchase_amount

Creating Age Group Column

A new column called **age_group** was created using quartile segmentation:

- Young Adult
- Adult
- Middle-aged
- Senior

This helps in analyzing revenue contribution across different age groups.

Creating Purchase Frequency Column

A new column **purchase_frequency_days** was created by converting textual purchase frequencies into numeric values.

Example conversions:

Weekly → 7 days
Monthly → 30 days
Quarterly → 90 days
Annually → 365 days

Removing Redundant Columns

The columns **discount_applied** and **promo_code_used** contained similar information.

Therefore, the **promo_code_used column was removed** to eliminate redundancy.

DATA CLEANING

```
[1]: import pandas as pd
     df = pd.read_csv('Dataset/customer_shopping_behavior.csv')
```

```
[2]: df.head(10)
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31
5	6	46	Male	Sneakers	Footwear	20	Wyoming	M	White	Summer	2.9	Yes	Standard	Yes	Yes	14
6	7	63	Male	Shirt	Clothing	85	Montana	M	Gray	Fall	3.2	Yes	Free Shipping	Yes	Yes	49
7	8	27	Male	Shorts	Clothing	34	Louisiana	L	Charcoal	Winter	3.2	Yes	Free Shipping	Yes	Yes	19
8	9	26	Male	Coat	Outerwear	97	West Virginia	L	Silver	Summer	2.6	Yes	Express	Yes	Yes	8
9	10	57	Male	Handbag	Accessories	31	Missouri	M	Pink	Spring	4.8	Yes	2-Day Shipping	Yes	Yes	4

```
[3]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   str
 3   Item Purchased          3900 non-null   str
 4   Category                3900 non-null   str
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   str
 7   Size                    3900 non-null   str
 8   Color                   3900 non-null   str
 9   Season                  3900 non-null   str
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   str
 12  Shipping Type           3900 non-null   str
 13  Discount Applied        3900 non-null   str
 14  Promo Code Used         3900 non-null   str
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   str
 17  Frequency of Purchases  3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 548.6 KB
```

```
[4]: df.describe(include='all')
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900	3900	3900	3863.000000	3900	3900	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	4	25	4	NaN	2	6	2	2
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	M	Olive	Spring	NaN	No	Free Shipping	No	No
freq	NaN	NaN	2652	171	1737	NaN	96	1755	177	999	NaN	2847	675	2223	2223
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN	NaN	NaN	3.750065	NaN	NaN	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN	NaN	NaN	0.716983	NaN	NaN	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN	NaN	NaN	2.500000	NaN	NaN	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN	NaN	NaN	3.100000	NaN	NaN	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN	NaN	NaN	3.800000	NaN	NaN	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN	NaN	NaN	4.400000	NaN	NaN	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN	NaN	NaN	5.000000	NaN	NaN	NaN	NaN

```
[5]: df.isnull().sum()

[5]: Customer ID              0
     Age                      0
     Gender                   0
     Item Purchased           0
     Category                 0
     Purchase Amount (USD)    0
     Location                 0
     Size                     0
     Color                    0
     Season                   0
     Review Rating           37
     Subscription Status      0
     Shipping Type            0
     Discount Applied         0
     Promo Code Used          0
     Previous Purchases       0
     Payment Method           0
     Frequency of Purchases   0
     dtype: int64

[6]: #Adding median of each category into the missing review of those category data
     df['Review Rating'] = df.groupby('Category')['Review Rating'].transform(lambda x: x.fillna(x.median()))

[7]: df.isnull().sum()

[7]: Customer ID              0
     Age                      0
     Gender                   0
     Item Purchased           0
     Category                 0
     Purchase Amount (USD)    0
     Location                 0
     Size                     0
     Color                    0
     Season                   0
     Review Rating            0
     Subscription Status      0
     Shipping Type            0
     Discount Applied         0
     Promo Code Used          0
     Previous Purchases       0
     Payment Method           0
     Frequency of Purchases   0
     dtype: int64
```

```
[8]: #Converting the column headers to snake casing (Customer Id:customer_id)
     df.columns = df.columns.str.lower()
     df.columns = df.columns.str.replace(' ','_')
     df = df.rename(columns={'purchase_amount_(usd)':'purchase_amount'})

[9]: df.columns

[9]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
            'purchase_amount', 'location', 'size', 'color', 'season',
            'review_rating', 'subscription_status', 'shipping_type',
            'discount_applied', 'promo_code_used', 'previous_purchases',
            'payment_method', 'frequency_of_purchases'],
           dtype='str')

[10]: # creating a column age_group
      labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
      df['age_group'] = pd.qcut(df['age'],q=4, labels = labels)

[11]: df[['age','age_group']].head(10)
```

[11]:

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
5	46	Middle-aged
6	63	Senior
7	27	Young Adult
8	26	Young Adult
9	57	Middle-aged

```
[12]: # create column purchase_frequency_days

      frequency_mapping = {
          'Fortnightly':14,
          'Weekly':7,
          'Monthly':30,
          'Quarterly':90,
          'Bi-Weekly':14,
          'Annually':365,
          'Every 3 Months':90
      }

      df['purchase_frequency_days'] = df['frequency_of_purchases'].map(frequency_mapping)

[13]: df[['purchase_frequency_days','frequency_of_purchases']].head(10)
```

[13]:

	purchase_frequency_days	frequency_of_purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

```
[14]: df[['discount_applied', 'promo_code_used']].head(10)
```

[14]:

	discount_applied	promo_code_used
0	Yes	Yes
1	Yes	Yes
2	Yes	Yes
3	Yes	Yes
4	Yes	Yes
5	Yes	Yes
6	Yes	Yes
7	Yes	Yes
8	Yes	Yes
9	Yes	Yes

```
[15]: (df['discount_applied'] == df['promo_code_used']).all()

[15]: np.True_

[16]: # Since discount_applied and promo_code_used are redundant we are droping one column

      df = df.drop('promo_code_used', axis=1)

[17]: df.columns

[17]: Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
             'purchase_amount', 'location', 'size', 'color', 'season',
             'review_rating', 'subscription_status', 'shipping_type',
             'discount_applied', 'previous_purchases', 'payment_method',
             'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
            dtype='str')
```

Fig 2: Data cleaning in jupyter notebook

DATA STORAGE IN PostgreSQL

After data preprocessing, the cleaned dataset was loaded into a PostgreSQL database using SQLAlchemy.

Steps performed:

1. Established connection to PostgreSQL database.
2. Created a database named **customer_behaviour**.
3. Loaded the cleaned dataset into a table called **customer**.
4. Verified data insertion using SQL queries.

```
[18]: pip install psycopg2-binary sqlalchemy

Requirement already satisfied: psycopg2-binary in c:\users\badig\appdata\local\programs\python\python313\lib\site-packages (2.9.11)
Requirement already satisfied: sqlalchemy in c:\users\badig\appdata\local\programs\python\python313\lib\site-packages (2.0.46)
Requirement already satisfied: greenlet>=1 in c:\users\badig\appdata\local\programs\python\python313\lib\site-packages (from sqlalchemy) (3.3.1)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\badig\appdata\local\programs\python\python313\lib\site-packages (from sqlalchemy)
(4.15.0)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.0.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

[19]: from sqlalchemy import create_engine
# Step 1: Connect to PostgreSQL
# Replace placeholders with your actual details

username = 'postgres'  #default user
password = 'YnbAn$4000?1$$' #the password you set during installation  $$@ means @
host = 'localhost' #if running locally
port = '5432' #default PostgreSQL port
database = 'customer_behaviour'  #database created in pgadmin

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

#step 2: Load DataFrame into postgresql

table_name = 'customer'  #choose any table name
df.to_sql(table_name, engine, if_exists='replace', index=False)

print(f"Data successfully loaded into table '{table_name}' in database '{database}'.")

Data successfully loaded into table 'customer' in database 'customer_behaviour'.
```

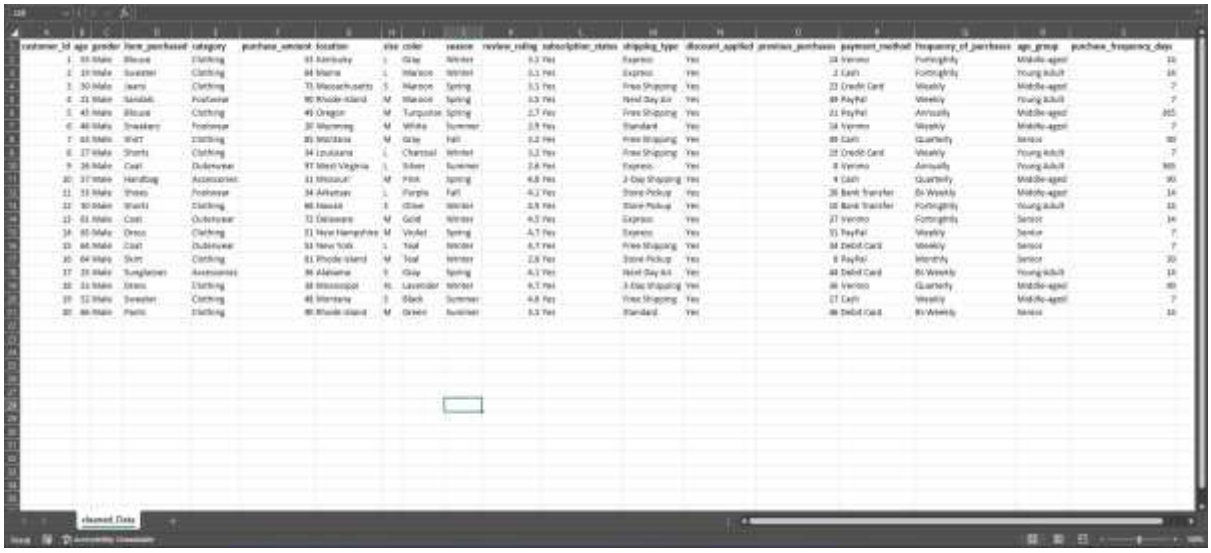


Fig 3: Cleaned Data

SQL BUSINESS ANALYSIS

Several SQL queries were executed to answer key business questions.

Examples include:

Revenue by Gender

The total revenue generated by male and female customers was calculated.

Discount Impact on Spending

Customers who used discounts but still spent above the average purchase amount were identified.

Top Rated Products

Products with the highest average review ratings were identified.

Shipping Type Analysis

Average purchase amounts were compared between **Standard** and **Express shipping**.

Customer Subscription Analysis

The spending behavior of subscribed and non-subscribed customers was analyzed.

Customer Segmentation

Customers were classified into:

- New Customers
- Returning Customers
- Loyal Customers

based on their **previous purchase count**.

Query | Query History

```sql
SELECT
    *
FROM
    CUSTOMER
LIMIT
    20;

--Q1. What is the totle revenue generated by male vs female customers ?
SELECT
    GENDER,
    SUM(PURCHASE_AMOUNT) AS REVENUE
FROM
    CUSTOMER
GROUP BY
    GENDER;

--Q2. Which customer used a discount but still spent more than the average purchase amount
SELECT
    CUSTOMER_ID,
    PURCHASE_AMOUNT
FROM
    CUSTOMER
WHERE
    DISCOUNT_APPLIED = 'Yes'
    AND PURCHASE_AMOUNT >= (
        SELECT
            AVG(PURCHASE_AMOUNT)
        FROM
            CUSTOMER
    );
```

Query | Query History

```sql
--Q3. Which are the top 5 products with the highest average review rating
SELECT
    ITEM_PURCHASED,
    ROUND(AVG(REVIEW_RATING::NUMERIC), 2) AS "Average Product Rating"
FROM
    CUSTOMER
GROUP BY
    ITEM_PURCHASED
ORDER BY
    AVG(REVIEW_RATING) DESC
LIMIT
    5;

--Q4. Compare the average Purchase Amounts between standard and express shipping
SELECT
    SHIPPING_TYPE,
    ROUND(AVG(PURCHASE_AMOUNT), 2)
FROM
    CUSTOMER
WHERE
    SHIPPING_TYPE IN ('Standard', 'Express')
GROUP BY
    SHIPPING_TYPE;

--Q5. Do subscribed customers spend more? Compare average spend and total revenue between subscriber and non-subscribers.
SELECT
    SUBSCRIPTION_STATUS,
    COUNT(CUSTOMER_ID) AS TOTAL_CUSTOMERS,
    ROUND(AVG(PURCHASE_AMOUNT), 2) AS AVG_SPEND,
    ROUND(SUM(PURCHASE_AMOUNT), 2) AS TOTAL_REVENUE
FROM
    CUSTOMER
GROUP BY
    SUBSCRIPTION_STATUS
ORDER BY
    TOTAL_REVENUE,
    AVG_SPEND DESC;
```

Query Query History

```sql
--Q6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT
    ITEM_PURCHASED,
    ROUND(
        100 * SUM(
            CASE
                WHEN DISCOUNT_APPLIED = 'Yes' THEN 1
                ELSE 0
            END
        ) / COUNT(*),
        2
    ) AS DISCOUNT_RATE
FROM
    CUSTOMER
GROUP BY
    ITEM_PURCHASED
ORDER BY
    DISCOUNT_RATE DESC
LIMIT
    5;
```

Query Query History

```sql
--Q7 Segment customers into new, returning, and loyal based on their total number of previous purchases, and show the count of each segment.
WITH
    CUSTOMER_TYPE AS (
        SELECT
            CUSTOMER_ID,
            PREVIOUS_PURCHASES,
            CASE
                WHEN PREVIOUS_PURCHASES = 1 THEN 'New'
                WHEN PREVIOUS_PURCHASES BETWEEN 2 AND 10  THEN 'Returning'
                ELSE 'loyal'
            END AS CUSTOMER_SEGMENT
        FROM
            CUSTOMER
    )
SELECT
    CUSTOMER_SEGMENT,
    COUNT(*) AS "Number of Customer"
FROM
    CUSTOMER_TYPE
GROUP BY
    CUSTOMER_SEGMENT;
```

Query Query History

```sql
--Q8, what are the top 3 most purchased products within each category
WITH
    ITEM_COUNTS AS (
        SELECT
            CATEGORY,
            ITEM_PURCHASED,
            COUNT(CUSTOMER_ID) AS TOTAL_ORDERS,
            ROW_NUMBER() OVER (
                PARTITION BY
                    CATEGORY
                ORDER BY
                    COUNT(CUSTOMER_ID) DESC
            ) AS ITEM_RANK
        FROM
            CUSTOMER
        GROUP BY
            CATEGORY,
            ITEM_PURCHASED
    )
SELECT
    ITEM_RANK,
    CATEGORY,
    ITEM_PURCHASED,
    TOTAL_ORDERS
FROM
    ITEM_COUNTS
WHERE
    ITEM_RANK <= 3;

--Q9 Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe?
SELECT
    SUBSCRIPTION_STATUS,
    COUNT(CUSTOMER_ID) AS REPEAT_BUYERS
FROM
    CUSTOMER
WHERE
    PREVIOUS_PURCHASES > 5
GROUP BY
    SUBSCRIPTION_STATUS;
```

```sql
--Q10 WHAT IS THE REVENUE CONTRIBUTION OF EACH AGE GROUP?
SELECT
    AGE_GROUP,
    SUM(PURCHASE_AMOUNT) AS TOTAL_REVENUE
FROM
    CUSTOMER
GROUP BY
    AGE_GROUP
ORDER BY
    TOTAL_REVENUE DESC;
```

Fig 4: SQL Queries in pgAdmin

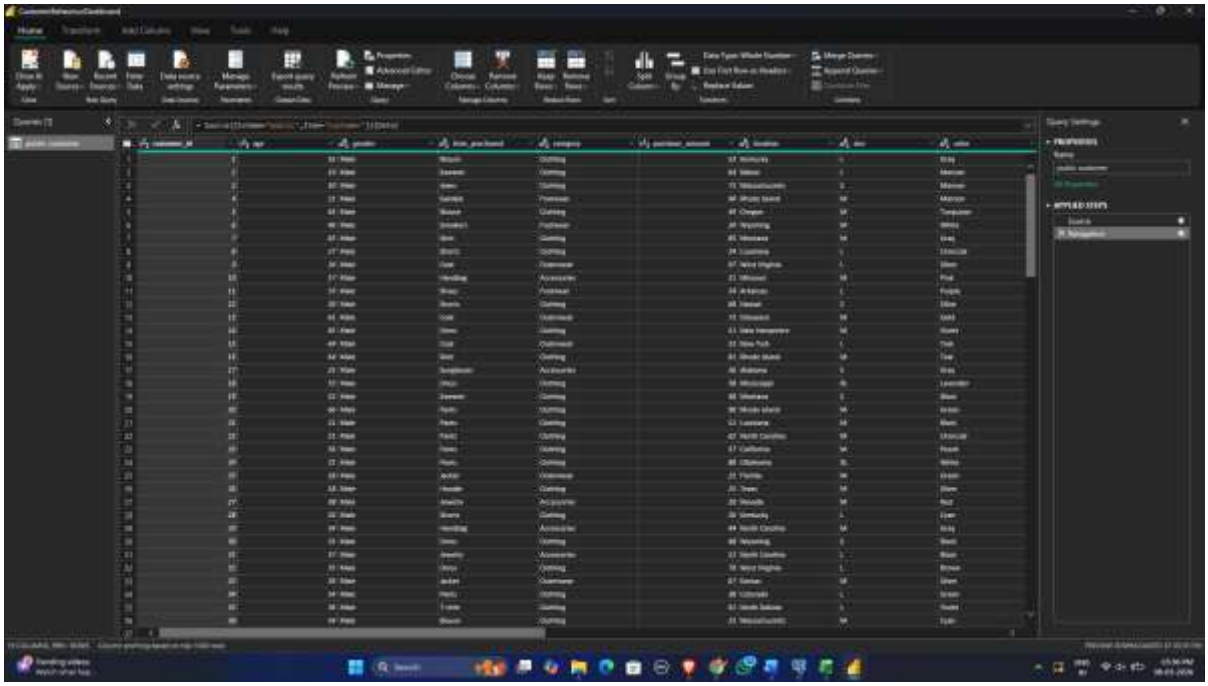
POWER BI DASHBOARD DEVELOPMENT

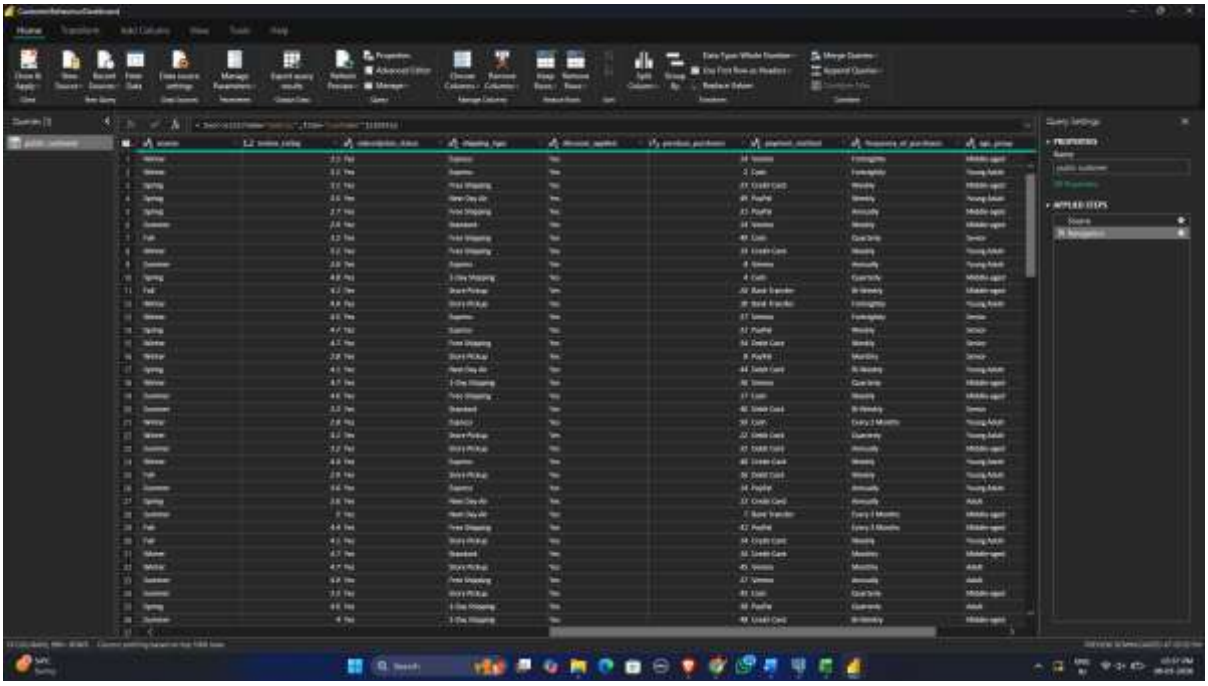
Power BI was used to build an interactive dashboard to visualize customer purchasing patterns.

The dashboard includes multiple visualizations such as:

- Revenue distribution
- Average purchase amount
- Customer count
- Review rating analysis
- Product category performance
- Age group revenue contribution

Power Query was used to import and transform data before visualization.





DAX MEASURES CREATED

Several measures were created using DAX for analytical calculations.

Examples include:

Average Purchase Amount

```
Average Purchase Amount = AVERAGE (customer[purchase_amount])
```

Average Review Rating

```
Average Review Rating = AVERAGE (customer[review_rating])
```

Number of Customers

```
Number of Customers = DISTINCTCOUNT (customer[customer_id])
```

These measures help calculate key performance indicators used in the dashboard.

DASHBOARD VISUALIZATIONS

The Power BI dashboard contains multiple visual components including:

- Bar charts
- Pie charts
- KPI cards
- Category-wise analysis
- Revenue breakdown
- Customer segmentation analysis

These visualizations provide quick insights into customer purchasing behaviour.

Fig 7: Power BI Dashboard

KEY INSIGHTS DERIVED

Several valuable insights were discovered through the analysis:

1. Certain product categories generate significantly higher revenue.
2. Customers using subscriptions tend to contribute a large portion of total revenue.
3. Some products receive consistently higher review ratings.
4. Discount usage influences purchasing patterns.
5. Loyal customers contribute significantly to repeat purchases.

These insights can help businesses optimize product offerings and marketing strategies.

CHALLENGES FACED

During the project, several challenges were encountered:

- Handling missing values in the dataset.
- Ensuring consistent column naming conventions.
- Removing redundant columns.
- Integrating multiple tools including Python, PostgreSQL, SQL, and Power BI.
- Creating meaningful visualizations for business insights.

These challenges were overcome through systematic data preprocessing and analytical techniques.

CONCLUSION

This project successfully demonstrates the use of modern data analytics tools to analyse customer shopping behaviour.

By combining Python, SQL, PostgreSQL, and Power BI, raw customer data was transformed into meaningful insights and interactive visualizations.

The results of this analysis can help businesses better understand customer preferences, improve product strategies, and enhance customer satisfaction.

FUTURE SCOPE

Future improvements for this project may include:

- Integrating machine learning models for customer prediction.
- Implementing real-time data analytics.
- Expanding the dataset with more customer attributes.
- Building automated reporting systems.

These improvements can further enhance the analytical capabilities of the system.

REFERENCES

- Python Pandas Documentation
- PostgreSQL Documentation
- Microsoft Power BI Documentation
- Data Analytics Learning Resources